PL-7008 Tailored Training: Trainer Answer Guide and One-Day Use Case

Create agents with the GitHub Copilot harness in Microsoft Copilot Studio

Prepared for	Neeraj Kumar, Sr Technical Trainer
Version	October 2026, aligned to the PL-7008 deck (GitHub Copilot harness edition)

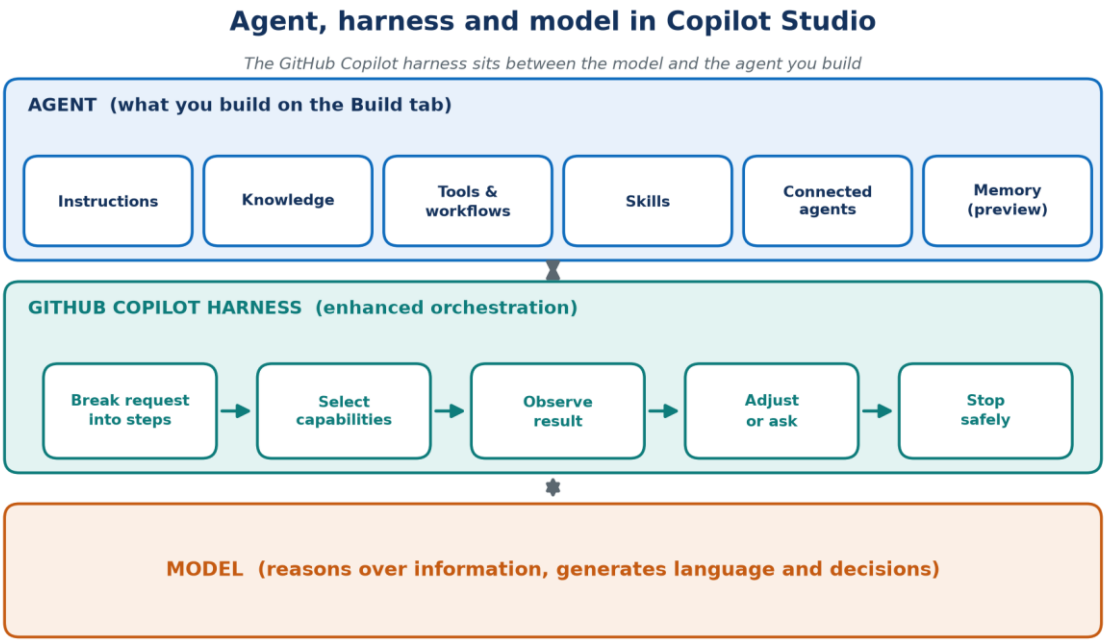
How to use this guide

This guide gives you ready answers for the tailored requests raised for PL-7008, and a one-day use case that lets students practise those answers hands-on. Everything is framed on the GitHub Copilot harness, the harness the current PL-7008 deck teaches, so the answers match what students will see on the Build, Preview, Evaluate and Monitor tabs.

The guide has four parts. Part A holds trainer answers for every request rated Highly possible or May be, with a diagram for each major concept.

What changed with the GitHub Copilot harness

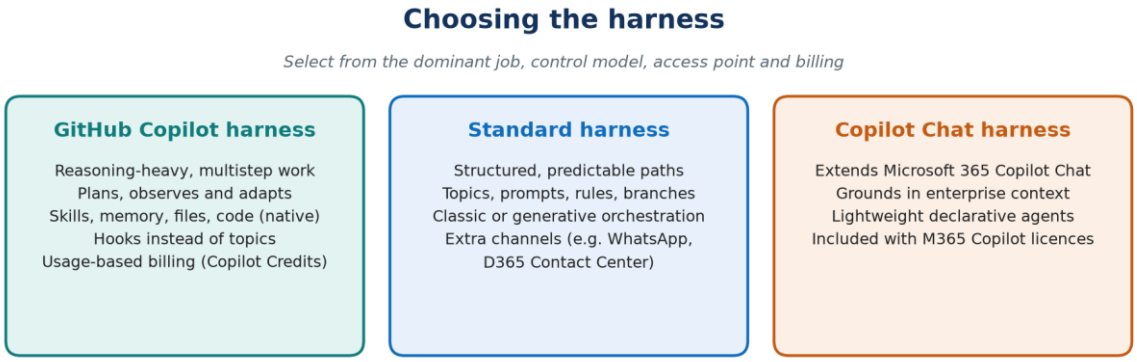
Many of the tailored questions were written with the older Standard harness in mind. Before answering them, anchor the class on how the GitHub Copilot harness changes the authoring model. Instead of authoring topics, flows and branching logic, makers describe the agent in natural language and connect the components it needs; the enhanced orchestration runtime then decides when to use knowledge, tools and skills.



The differences below come up repeatedly in the tailored questions, so keep this table to hand. It reflects the Microsoft Learn comparison of Standard and GitHub Copilot harness features (page last updated 20 October 2026).

Area	Standard harness	GitHub Copilot harness	Why it matters in class
Prompt tools	Supported	Replaced by Skills	Teach reusable task logic as skills (SKILL.md)
Topics and system triggers	Supported	Replaced by Hooks	Hooks run workflows at lifecycle points, such as session start or before a tool runs
Azure AI Search knowledge	Supported	Not supported	Advanced retrieval tuning questions point to Foundry, not the harness
Dataverse, SharePoint, files, websites, Graph connectors	Supported	Supported	Core knowledge options are unchanged
Connectors, MCP, workflows	Supported	Supported	Integration answers apply to both
Skills, memory	Not supported	Supported	Unique to the GitHub Copilot harness
Code interpreter	Opt-in	Native	Useful for calculations and file work
Authentication	Entra ID, none, manual	Entra ID, none	Manual or custom authentication is not supported
Channels	Teams, M365, web, SharePoint, WhatsApp, D365 Contact Center	Teams, M365, web	Channel needs can decide the harness
Languages	Added and enabled explicitly	Out of the box; response language set by instructions	Multi-language design moves into instructions
Billing	Licence-included or consumption	Always usage-based (Copilot Credits)	Preview, build and evaluation consume credits

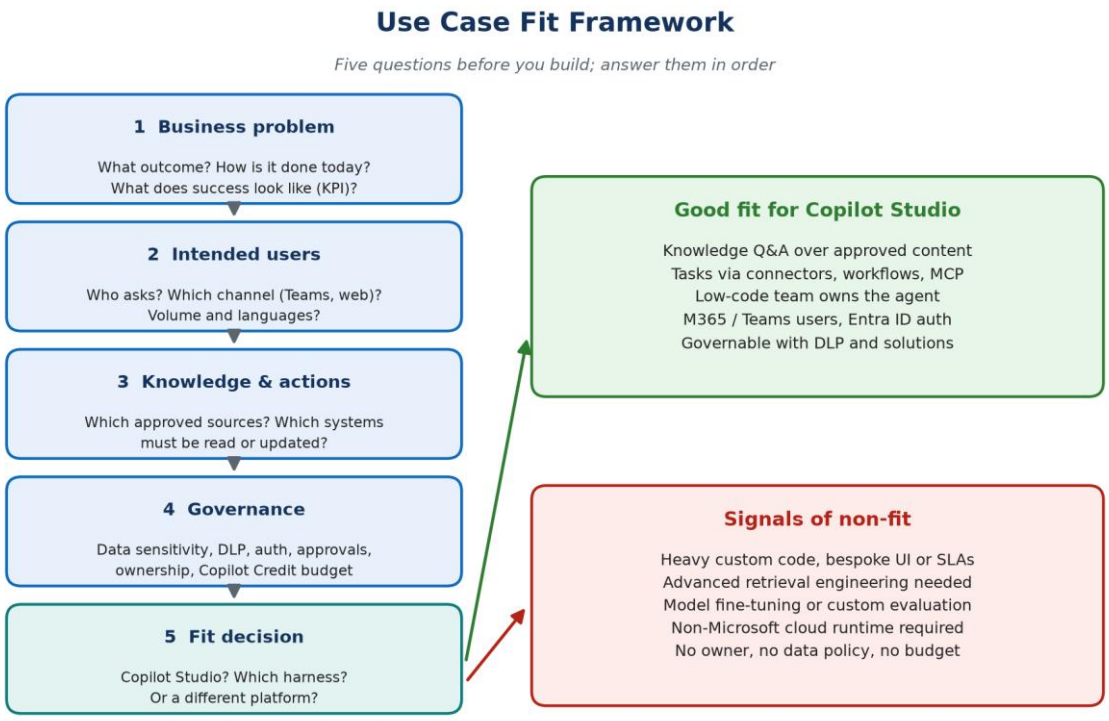
Trainer tip: if a learner asks whether data goes to GitHub, Microsoft states that the harness is a Copilot Studio framework, not the GitHub Copilot service, and that customer data is not sent to or processed by the GitHub Copilot service when agents run in Copilot Studio.



Part A: Trainer answers, Highly possible

2. Use Case Fit Framework

Short answer: Before building, run five questions in order: the business problem, the intended users, the knowledge and actions required, the governance constraints, and finally the fit decision. Only the last question names a platform; the first four collect evidence for it.



Use the worksheet below as the class handout. It maps directly onto the deck's Define purpose and scope slide, so students recognise it later when they write instructions.

Dimension	Question to answer	Evidence to capture
Business problem	What outcome do we want and how is it handled today?	Current volume, cost or time per request, target KPI
Intended users	Who asks, through which channel, in which languages?	Personas, channel (Teams, web), language list, peak volume
Knowledge required	Which approved sources answer the questions?	Owner, format, update frequency, sensitivity of each source
Actions required	Which systems must be read or updated?	System, action, connector or MCP availability, need for confirmation
Governance	What controls apply?	Data classification, DLP policy, authentication, approvals, owner, credit budget
Fit decision	Is Copilot Studio right, and which harness?	Decision, harness, risks, what stays out of scope

Talk track: Most failed agent projects skip question one and start in the tool. The framework keeps the conversation on the job to be done, and it gives you a natural moment to ask whether a simpler answer exists, such as a SharePoint page or a single workflow.

5. When NOT to use Copilot Studio

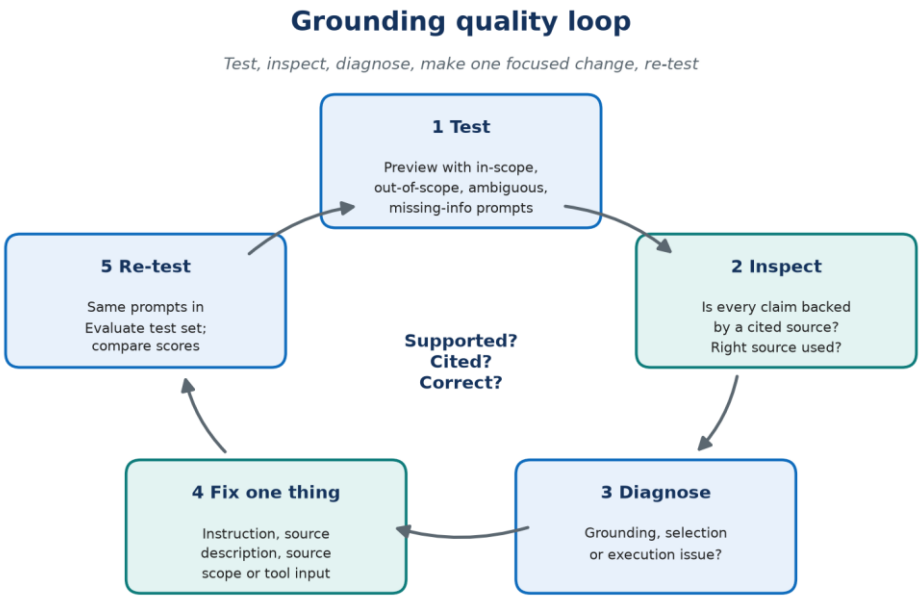
Short answer: PL-7008 teaches low-code agent creation by makers. When a requirement needs heavy custom code, bespoke user interfaces, model training, advanced retrieval engineering or a non-Microsoft runtime, Copilot Studio is the wrong primary tool, even if it can still play a part.

Scenario	Fit?	Better route and reason
Employees ask policy questions in Teams	Fit	Copilot Studio; consider the Copilot Chat harness if no actions are needed
Agent reviews invoices, uses several tools and adapts when an invoice does not match the PO	Fit	GitHub Copilot harness: reasoning-heavy, multistep work
Customer bot on WhatsApp with fixed menu-style paths	Partial	Standard harness: WhatsApp is not a GitHub Copilot harness channel
Same approval steps every time, no conversation	Non-fit as an agent	A workflow on its own; no reasoning is needed
Custom vector index with tuned ranking and chunking	Non-fit	Microsoft Foundry; Azure AI Search knowledge is not supported in the harness
Branded app with custom UI, strict latency SLAs and bespoke code	Non-fit	Custom development, for example Agent Framework
No owner, no data policy, no credit budget	Not yet	Fix governance first; any platform will fail

Talk track: Saying no to the wrong use case protects credibility. Frame it as choosing the smallest platform that meets the requirement, the same principle the deck uses for choosing capabilities.

9. Grounding quality assessment

Short answer: A grounded answer is one where every claim can be traced to a cited, approved source. Assess it by running representative prompts, inspecting citations, classifying any failure as a grounding, selection or execution issue, making one focused change, and repeating the same prompts.



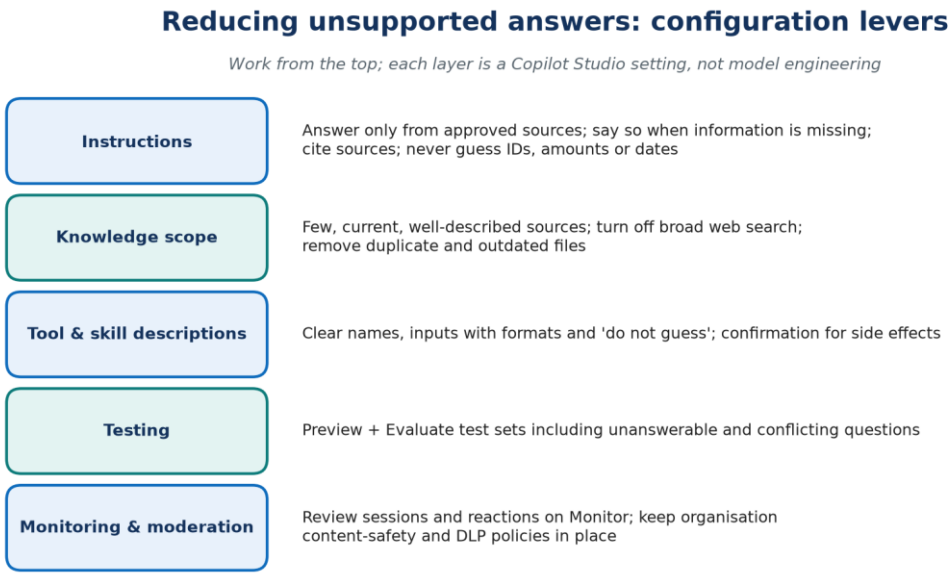
Score every test response on three checks and record the result. This makes Preview exploration repeatable and prepares the prompts for an Evaluate test set later.

Check	Pass when	Typical failure
Supported	Every factual statement appears in a cited source	Answer adds plausible detail not in the source
Cited	Citation points to the right, current source	Old or duplicate file cited; no citation
Correct	Answer matches what the business owner expects	Right source, wrong section or misread threshold

Issue type	Symptom	First fix to try
Grounding	Wrong or missing facts, outdated citation	Check source availability, remove outdated content, tighten source instructions
Selection	Wrong source, tool or skill chosen	Improve names and descriptions, remove overlap, test near-matching requests
Execution	Tool or workflow fails or returns nothing	Inspect inputs and permissions, review connector or server errors, verify output

10. Reducing hallucinations

Short answer: At Copilot Studio level, unsupported answers are reduced through configuration, not model engineering: precise instructions, a small set of current and well-described sources, clear tool and skill descriptions, systematic testing, and monitoring after publishing.



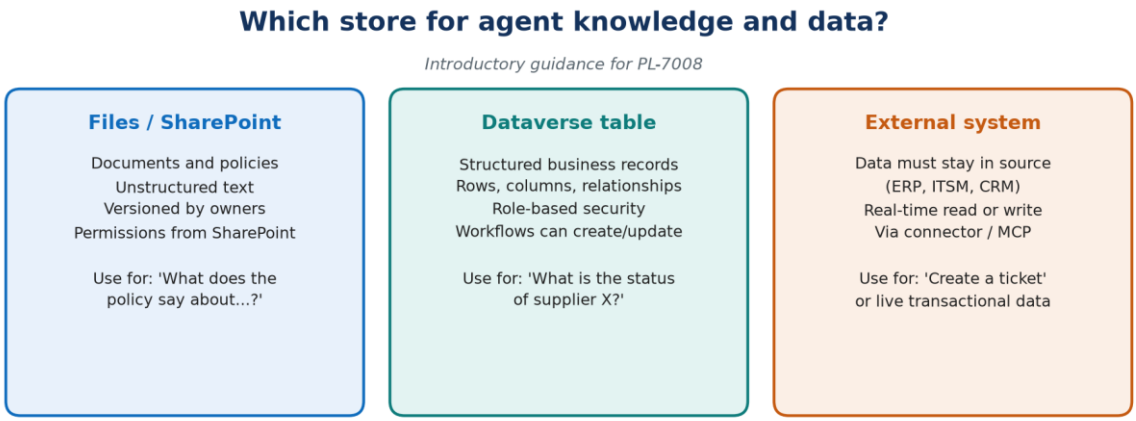
Give learners this instruction block as a pattern they can adapt. It is deliberately specific and testable.

```
Answer only from the configured knowledge sources.
If the sources do not contain the answer, say you could not find it
and suggest contacting procurement@contoso.com. Do not guess.
Cite the source for every policy statement.
Never invent supplier IDs, amounts, dates or approval decisions.
If two sources disagree, use the most recent version and say so.
```

- Turn off broad web search when the agent must answer only from organisational content.
- Remove duplicate and superseded files; conflicting versions are a leading cause of confident but wrong answers.
- Write tool inputs with formats and an explicit instruction not to guess missing values.
- Include unanswerable and conflicting questions in every test set, not only happy-path prompts.
- After publishing, review sessions, reactions and unsupported answers on the Monitor tab and keep organisational content-safety and DLP policies in place.

19. Dataverse: when and how to use it

Short answer: Use Dataverse when the agent needs structured business records with rows, columns, relationships and role-based security, for example a supplier register. Use files or SharePoint for documents and policies, and leave transactional data in its source system when it must stay there.



Keep design introductory. The five rules below are enough for a maker to build a table an agent can use well; full data modelling belongs in a Dataverse course.

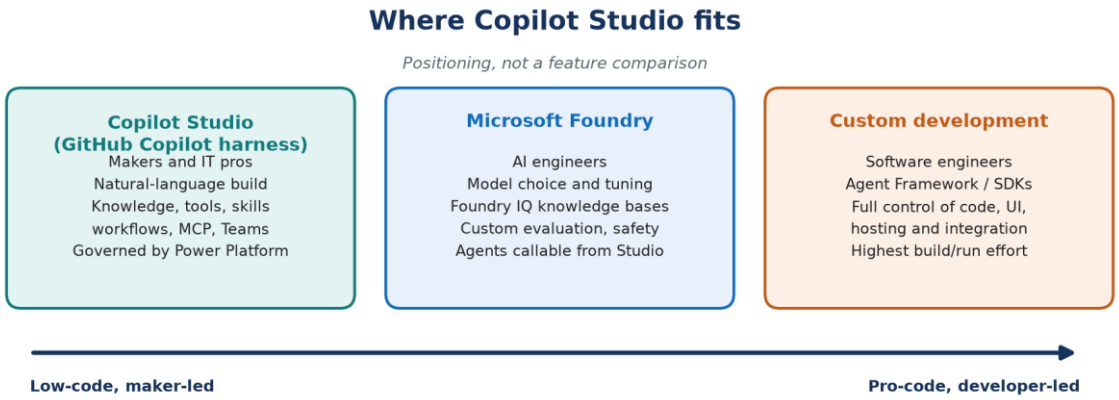
Design rule	Why it helps the agent
Give the table and every column a clear display name and description	The orchestrator relies on names and descriptions to pick the right data
Use a meaningful primary name column (for example Supplier Name)	Answers and citations read naturally
Use choice columns for statuses (Approved, Conditional, Blocked)	Consistent values make filtering and answers reliable
Keep only columns the agent needs; add a business key such as Supplier ID	Smaller schemas reduce selection mistakes
Secure with security roles; test as a normal user	Agents respect the user's access, so Preview as maker can hide permission gaps

How in the harness: On the Build tab, add knowledge, choose Dataverse, select the table, then describe what the table is for. Dataverse tables are supported knowledge in both harnesses. Workflows can create or update Dataverse rows when the agent must write data.

Part A: Trainer answers, May be

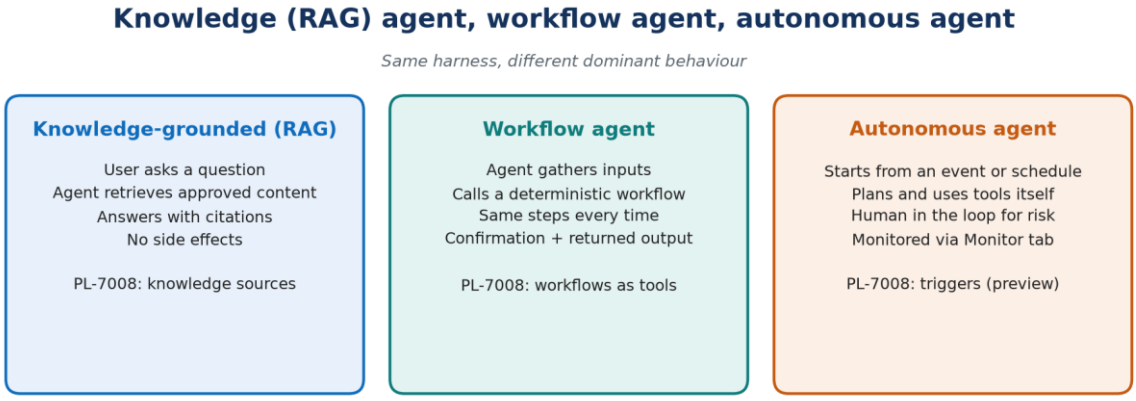
Treat each item as a 5 to 10 minute positioning discussion. Do not promise hands-on depth beyond what the labs include.

1. Copilot Studio vs Microsoft Foundry vs custom development



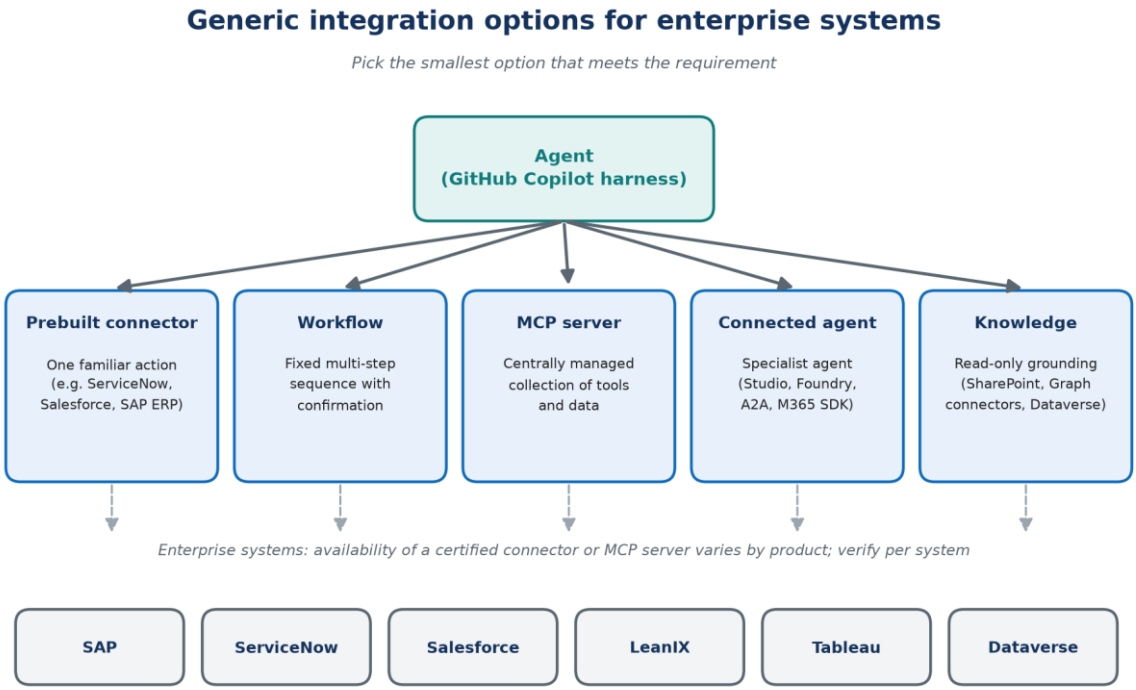
Position by who builds and how much control is needed. Copilot Studio suits makers who describe agents in natural language and connect governed components. Foundry suits AI engineers who need model choice, tuned knowledge bases and custom evaluation; its agents and Foundry IQ knowledge can be connected into Copilot Studio. Custom development, for example with Microsoft Agent Framework, suits engineering teams that need full control of code, UI and hosting. The three are complementary, not competing.

3. RAG vs workflow agent vs autonomous agent



All three can run on the GitHub Copilot harness; what differs is the dominant behaviour. A knowledge-grounded agent retrieves approved content and answers with citations. A workflow agent hands a fixed sequence to a deterministic workflow, with confirmation and returned output. An autonomous agent starts from an event or schedule, using triggers (preview) or an agent node in a workflow, and plans its own tool use with human checkpoints for risky steps. In the lab, ProcureAssist is a knowledge agent that becomes a workflow agent in Lab 6.

6. Integration options for SAP, ServiceNow, Salesforce, LeanIX and Tableau



Teach the generic options once and apply them to each system, rather than six architectures. The rule is the deck's: choose the smallest capability that meets the requirement.

System	Typical agent need	Generic option to discuss
ServiceNow	Create or look up a ticket	Prebuilt connector as a tool, or a workflow when steps are fixed
Salesforce	Read account or opportunity data	Prebuilt connector; confirmation before any update
SAP	Check PO or invoice status	SAP connector through a workflow, usually built with IT
LeanIX	Look up application or ownership data	Check the connector catalogue; otherwise a custom connector or vendor MCP server
Tableau	Point users to the right dashboard	Usually knowledge or a link; check catalogue for an action connector

Availability of certified connectors and MCP servers changes over time and by licence. Verify in the target tenant before promising an option.

8. Retrieval tuning, configuration level

Within Copilot Studio, tuning means improving what the agent can retrieve and how it is told to use it. Name and describe each source for its intended use, narrow sources to the relevant site or folder, remove duplicates, restrict broad web search, refine instructions about which source wins, and compare results with the same Evaluate test set before and after each change. Ranking, indexing parameters and vector design are outside scope and live in Foundry.

11. Large enterprise knowledge repositories

- Start from the questions, not the repository: select the libraries that answer the top 20 questions.
- Prefer a curated, owned set of current documents over an entire intranet.

- Describe each source clearly so the orchestrator can choose between them.
- Remember that users only see content they already have permission to read; test with real user accounts.
- Enterprise-scale ingestion, performance and repository architecture need a separate workshop.

12. Multi-language knowledge design

In the GitHub Copilot harness, languages work out of the box and the response language is controlled by the agent instructions. That moves design into two places: an instruction such as respond in the language of the user's question while keeping citations to the original source, and a test set that includes each supported language. Keep one authoritative version of each policy; if translated versions exist, describe each source with its language so the agent picks consistently. Lab 3 includes a short Hindi and French test.

13. Custom connectors

A custom connector wraps a REST API, usually from an OpenAPI description, so it can be used like any prebuilt connector, including as an agent tool or inside a workflow. Explain when you need one (no prebuilt connector exists), who typically builds it (IT or a pro developer), and that it falls under the same DLP policies. Building, authenticating and troubleshooting one is a demo at most in a one-day course.

16. Authentication patterns

Pattern	What it means	Teach this
Microsoft Entra ID	Users sign in; knowledge and tools act within their permissions	Default for internal agents; test as a real user after publishing
No authentication	Anonymous access	Only for public, non-sensitive content
Manual or custom	Third-party identity providers	Not supported in the GitHub Copilot harness; use the Standard harness if required
Connection credentials	How a tool or workflow connects to a system	Decide whether the user's or a maker's connection is used; least privilege

17. MCP servers

The Model Context Protocol lets an agent discover a centrally managed collection of tools and data from a server's descriptions. In the harness you connect an existing MCP server as a tool; authentication and permissions still control access, and you validate tool selection, inputs, outputs and errors as with any tool. Use a direct connector when one familiar action is the simpler design. Position this as a trainer demo in Lab 5, not a hands-on module.